

Assignment 5 - Capstone Project Proposal

James Walden

DS552 - Generative AI

1. Title

The title of my project will be:

Integrating Machine Learning Recommendations with Conversational LLM Control for Adaptive Meal Planning

2. Introduction

Existing meal planning tools often rely on static templates or simple filtering mechanisms that fail to account for the dynamic and personalized nature of real-world constraints. As a result, people are forced to manually track trade-offs between nutrition, convenience, cost, and preference. This disconnect highlights the need for a more adaptive and intelligent system capable of synthesizing multiple constraints simultaneously while remaining intuitive and user-friendly.

The purpose of the project is to develop a web application to simplify the process of meal planning, finding optimized meal plans that adhere to a myriad of constraints and recommend meals that maximize user satisfaction. By using a mix of machine learning algorithms for meal-recommendation systems with conversational large language models for a user-friendly interface for the application. The application will provide users with a meal recommendation system that remembers previous preferences, creates weekly meal plans dynamically adapting based on user prompts, and enables iterative refinement of those plans through natural language interaction. Users will be able to request substitutions (e.g., “make Wednesday vegetarian”), adjust caloric targets, remove allergens, or optimize for time and budget constraints. The system will update the weekly planning interface in real time by parsing structured outputs generated by the large language model.

3. Model Selection

The model selected for this application is Mistral 7B v0.1, developed by Mistral AI. The model was chosen due to its strong performance-to-parameter efficiency ratio and its demonstrated reasoning capabilities relative to similarly sized open-source models. Reported benchmark evaluations indicate competitive performance compared to larger models such as LLaMA 2 13B, while maintaining lower computational overhead.

For the scope of this project, a model is needed that can interpret user constraints, generate structured output, and perform multi-step reasoning with dietary restrictions and goals. The model will be accessed through Hugging Face’s hosted API, allowing for real-time

interaction without requiring local GPU resources (which I do not have access to as I am using a Mac). This approach enables reproducible evaluation and seamless integration with the web-based system while keeping hardware and cost requirements minimal.

4. Project Definition and Use Case

The proposed web application can be defined as a hybrid conversational meal-planning system that integrates predictive recommendation modeling with a structured large language model interface. At its core, the system functions as an interactive meal planning tool augmented by a conversational assistant capable of generating, modifying, and explaining meal plans based on stored user preferences.

The system can be divided into the following layers:

1. A front-end for viewing and interacting with generated meal plans
2. Authentication to enable persistent individualized user experiences
3. A database to store user preferences, restraints, and historic performance
4. A Machine Learning Model used for meal or ingredient recommendation
5. A LLM acting as the main user interface for reasoning, interpreting user prompts, and generated structured data

The large language model serves as the primary interaction layer between the user and the system. Unlike traditional chatbots that produce only free-form text responses, the LLM in this application will generate both conversational output and structured data. Specifically, the model will:

1. Interpret natural language constraints (e.g., dietary restrictions, caloric goals, ingredient substitutions)
2. Translate those constraints into structured outputs following a predefined JSON schema
3. Trigger updates to the application state (e.g., modifying a specific day's meals)
4. Provide explanations or suggestions to guide user decision-making

By constraining the model's outputs to a predetermined format, the system ensures that generated responses can be parsed programmatically and injected directly into the frontend interface. This allows the LLM not only to communicate with the user but also to dynamically modify the weekly meal plan displayed in the application.

This hybrid approach is effective because it clearly separates the responsibilities of the machine learning model and the large language model. The machine learning model focuses on generating strong meal or ingredient recommendations based on user preferences and constraints, while the LLM handles interpreting user prompts, reasoning through modifications, and generating structured outputs for the application. Trying to place both predictive

recommendation and conversational reasoning into a single model would increase complexity and reduce overall effectiveness. By allowing each model to specialize in what it does best, the system remains more efficient, modular, and easier to scale.

5. Implementation Plan

The application will be developed as a React web application with OAuth for user authentication and MongoDB for unstructured data storage, allowing persistent storage of user preferences and historical meal plans. The backend will be implemented in Node.js using Express, managing database interactions and API requests from the frontend. The machine learning and LLM functionalities will be implemented as Python microservices, with Node.js coordinating requests between the frontend and these AI services.

The application will be hosted on AWS, providing reliable computational resources and storage to support both the machine learning and LLM services. The machine learning model will be developed using Scikit-Learn. While it is not yet determined whether the task will be framed as a classification or regression problem, this will become clear as suitable datasets are identified. Potential datasets include USDA FoodData Central for minimally processed foods or a Food Recommendation System dataset from Kaggle.

The LLM component will utilize Mistral 7B, accessed via Hugging Face hosted API. The LLM will handle natural language interpretation of user prompts, multi-step reasoning over dietary constraints, and structured output generation to dynamically update the frontend interface.

The following must be completed in order to deploy the application (not in order):

1. Well-designed frontend with components capable of rendering structured data from the LLM
2. Login page with OAuth authentication
3. Backend routes for MongoDB CRUD operations to communicate with the frontend
4. Exploratory Data Analysis (EDA) and preprocessing of dataset
5. Machine learning model selection, training, and evaluation
6. API routes connecting the ML model, LLM, and database
7. Frontend interface for users to communicate naturally with the LLM
8. Provide context and constraints to the LLM for structured output generation
9. Parse LLM outputs and inject them into the frontend view dynamically
10. Deploy frontend and backend to AWS, ensuring services communicate correctly
11. Implement error handling and input validation to ensure reliable user experience
12. Evaluate LLM performance

6. Model Evaluation

In order to evaluate our model, several techniques will be applied to measure performance. Latency times will be important, given the multi-stage communication process of the system; ensuring users aren't waiting too long for a response is critical. Processing efficiency will also be tracked by monitoring CPU/GPU usage and average memory per query. ROUGE can be used to evaluate model performance, as it primarily measures recall, which is important for a system like this. Structural accuracy will be measured by the proportion of responses that match the specified output structure, and constraint satisfaction will track how often the model successfully meets all user-defined restrictions. Finally, for robustness, the model will be tested with multiple users simultaneously to evaluate its performance at scale, as well as with faulty or unexpected inputs to ensure it can handle edge cases.

7. Expected Outcome and Challenges

Completion of this project will lead to the successful deployment of an AI-driven service which aids in the process of meal planning. A successful outcome would feature an application able to reason over given constraints and user prompts to perform in an effective and expected way. The application should be easy to understand, and as such recommendations coming from the system should be reasonable.

There are several potential challenges in the implementation of this system. Having the separation of the recommendation system and reasoning model introduce more layers of complexity in which the system can fail. The separation also comes with a higher computational cost and potentially higher latency in response. Having the model directly able to interface with the application itself, rather than simply respond to user prompts, introduces additional complexities in the implementation of the front end and how user queries are handled.

Keeping these challenges in mind during development will be key to finding unique solutions specific to this application to improve performance of the overall system.

8. Resource Requirement

The development and deployment of the proposed application will require the following tools, frameworks, and datasets:

Frontend & Authentication:

React - for building the interactive web interface
Next.js - for server-side rendering and API routes
OAuth - for secure user authentication

Backend & Database:

Node.js with Express - to handle API requests and coordinate communication between the frontend, database, and AI services

MongoDB - for unstructured storage of user preferences, historical meal plans, ect

Machine Learning & Data Processing:

Python - primary language for ML and LLM services

Scikit-Learn - for developing and training the meal recommendation model

Pandas / NumPy / Matplotlib - for exploratory data analysis and preprocessing

Datasets:

USDA FoodData Central - minimally processed food samples and nutritional information

Kaggle Food Recommendation System dataset - dataset for meal preference prediction

Large Language Model (LLM):

Mistral 7B - accessed via Hugging Face API

Deployment & Cloud Infrastructure:

AWS - hosting for frontend, backend, and AI microservices

Additional Tools & Libraries:

FastAPI - for Python microservice endpoints

Git & GitHub - version control and collaboration

These resources collectively will enable development, deployment, and evaluation of the hybrid meal-planning system.

9. Conclusion

The proposed hybrid meal-planning system integrates machine learning, large language models, and modern web technologies to create a personalized and interactive dietary recommendation platform. By combining a structured machine learning model for preference prediction with an LLM capable of interpreting natural language constraints and generating structured outputs, the system leverages the strengths of both predictive modeling and contextual reasoning. The microservice-based architecture ensures modularity, allowing each component to operate independently while remaining fully integrated.

Successful implementation requires not only applied machine learning, but also system-level design considering scalability and robustness. By deploying the LLM through a

hosted API and maintaining lightweight local ML services, the system remains computationally feasible while preserving real-time interaction capabilities.

The potential impact of this application lies in its ability to provide adaptive, user-centered meal planning that accounts for evolving preferences, dietary restrictions, and long-term goals. Beyond meal planning, the architectural framework established in this project can be generalized to other personalized recommendation systems that require both statistical prediction and natural language reasoning. Hybrid AI systems are becoming increasingly common, and this project serves as a practical exploration of integrating structured machine learning models with generative reasoning models in a scalable web-based environment.